REVIEW – 2

QueryLang: A Compiler-Style Query Processing Engine

Compiler Design Laboratory Project (BCSE307P)

Name:	Aman Katakwar
Reg. No.:	24BCE0160
Course Code:	BCSE307P
Submitted to:	Prof. Ranjithkumar S
Milestone:	Review 2 — Core Implementation & Functional Prototype

School of Computer Science and Engineering | Academic Year 2026–27

1. Review 1 to Review 2 Transition

Progression from design specifications to a fully functioning compiler pipeline

Review 1 Deliverables (Phase 1: Design)

- Problem definition, scope, and domain constraints established.
- Formal Context-Free Grammar (CFG) in EBNF specified.
- 7-stage compiler architecture planned.
- Working Lexer prototype with tokenization and character diagnostics.
- Subsequent stages explicitly reserved for Phase 2.

Review 2 Deliverables (Phase 2: Implementation)

- **Complete Lexer:** Token recognition with precise line/col coordinates.
- **Recursive-Descent Parser:** LL(1) predictive grammar analysis.
- **Abstract Syntax Tree (AST):** Object-oriented tree nodes & visualizer.
- **Symbol Table / Catalog:** Scope management & type declarations.
- **Semantic Analyzer:** Name resolution and static type checking.
- **Relational Query Planner:** Intermediate Code (IR) generation.
- **Execution Engine:** In-memory relational interpretation.

2. Review 2 Rubric Alignment (25 Marks)

Strict compliance with the Compiler Design Laboratory Instruction Manual criteria

Evaluation Criteria	Marks	QueryLang Implementation & Evidence
Implementation Progress	**7**	100% of planned Phase 2 modules completed: Lexer → Parser → AST → Symbol Table → Semantic Analyzer → Query Planner → Execution Engine. Zero mock stages.
Technical Correctness	**5**	Deterministic token scanning, accurate LL(1) recursive-descent parsing, correct AST hierarchy, exact semantic validation, relational operator execution.
Compiler Concept Application	**4**	Direct application of regular expressions, CFGs, LL(1) predictive parsing, AST construction, symbol tables, static type inference, and intermediate code representation.
Code Quality	**3**	Clean modular architecture across 8 sub-packages, comprehensive docstrings, meaningful variable/function naming, pure Python 3 standard library (zero external dependencies).
Testing	**3**	Formal 16-test verification matrix + 20 PyUnit test cases covering positive, compound, error, and boundary conditions (100% pass rate in 0.004s).
Individual Understanding	**3**	In-depth comprehension of parsing algorithms, relational algebra translation, symbol table scope management, and runtime query interpretation.

3. Core Implementation Architecture & Data Flow

Modular, inspectable pipeline transforming declarative queries into executed results



Inter-Module Contracts and Data Representation

1. **Tokens:** Token stream with token type, evaluated literal value, line, and column coordinates.
2. **AST:** Hierarchical composite tree representing projections, table targets, and binary condition nodes.
3. **Symbol Table:** Schema Catalog validating table identifiers, attribute availability, and types.
4. **Logical Plan (IR):** Tree of relational algebra operators: SCAN → FILTER → PROJECT → SORT → LIMIT.
5. **Engine:** Tree-walking runtime interpreter executing plan against in-memory records to produce formatted ASCII tables.

4. Compiler Design Concepts Applied in Phase 2

Direct theoretical grounding in core language-processing fundamentals

Compiler Concept	Theoretical Foundation	QueryLang Implementation Module
Lexical Analysis	Regular expressions, DFA state scanning, token classification	`querylang/lexer/lexer.py`
Syntax Analysis	Context-Free Grammar (CFG), LL(1) predictive parsing	`querylang/parser/parser.py`
Abstract Syntax Tree	Hierarchical syntax tree representation and traversal	`querylang/ast/nodes.py`
Symbol Table	Scope resolution, identifier binding, attribute data types	`querylang/schema/catalog.py`
Semantic Analysis	Static semantic validation, type inference, type checking	`querylang/semantic/analyzer.py`
Intermediate Code (IR)	Relational algebra operator tree (σ, π, τ)	`querylang/planner/planner.py`
Runtime Execution	Tree-walking evaluation / interpreter on relation records	`querylang/execution/engine.py`
Diagnostic System	Contextual error reporting with line/col caret (^) pointers	`querylang/errors/errors.py`

5. Recursive-Descent Parser & Formal Grammar

Top-down predictive parsing implementing stratified operator precedence

QueryLang EBNF Grammar

```
query         ::= SELECT column_list
                  FROM table_name
                  [ where_clause ]
                  [ order_clause ]
                  [ limit_clause ] ";"
column_list   ::= "*" | col ("," col)*
where_clause  ::= WHERE condition
condition     ::= or_expr
or_expr       ::= and_expr (OR and_expr)*
and_expr      ::= comparison (AND comparison)*
comparison    ::= primary [ comp_op primary ]
primary       ::= col | literal | "(" cond ")"
```

Algorithmic Highlights

- **Elimination of Ambiguity:** Stratified grammar productions enforce strict operator precedence: `OR < AND < COMPARISONS < PARENTHESES`.
- **Predictive LL(1) Lookahead:** Employs `current_token` and `peek()` to deterministically choose production alternatives without backtracking.
- **Syntactic Error Recovery:** Mismatched tokens raise explicit `SyntaxError` reporting exact line, column, expected symbol, and offending token.

6. Abstract Syntax Tree (AST) Architecture

Strongly typed object hierarchy representing synthesized query structures

AST Node Classes (`nodes.py`)

- `SelectQueryNode`: Root node encapsulating projection list, table target, condition, sort, and limit.
- `ColumnNode`: Represents attribute references (e.g. `name`, `cgpa`).
- `TableNode`: Represents source relation identifiers (e.g. `students`).
- `LiteralNode`: Encapsulates constants and intrinsic types (`INTEGER`, `FLOAT`, `STRING`, `BOOLEAN`).
- `BinaryOpNode`: Encapsulates binary operators (`AND`, `OR`, `>=`, `=`, etc.).

Generated Visual Tree

```
SelectQueryNode
|-- Projected Columns: [name, cgpa]
|-- Source Table: students
|-- WHERE Condition:
    \-- BinaryOpNode(AND)
        |-- BinaryOpNode(>=)
        |   |-- ColumnNode(cgpa)
        |   \-- LiteralNode(8.5 : FLOAT)
        \-- BinaryOpNode(>)
            |-- ColumnNode(age)
            \-- LiteralNode(20 : INTEGER)
```

7. Symbol Table Management (Schema Catalog)

Centralized repository for relational schema, attribute scopes, and data types

Symbol Table Architecture

- **Relational Catalog:** In QueryLang, the traditional compiler symbol table is embodied by the `SchemaCatalog`.
- **Table Scope:** `TableSymbol` manages table identifiers and child column definitions.
- **Attribute Symbols:** `ColumnSymbol` tracks column identifiers, data types, and semantic constraints.
- **Lookup Methods:** Provides O(1) scope lookups: `has_table()`, `get_table()`, `has_column()`.

Default Preloaded Schema (`students`)

Column Identifier	Declared Type	Semantic Role
`id`	INTEGER	Primary key
`name`	STRING	Student full name
`age`	INTEGER	Age in years
`branch`	STRING	Academic department
`cgpa`	FLOAT	Cumulative GPA

8. Semantic Analysis & Type Diagnostics

Enforcing semantic correctness beyond grammatical validity

Semantic Validation Steps

- **Table Scope Check:** Ensures `FROM` relation exists in catalog.
- **Attribute Scope Check:** Ensures projected attributes and condition columns exist on the referenced relation.
- **Static Type Inference:** Traverses condition expression subtrees; resolves column types from symbol table and validates operator compatibility.
- **Type Compatibility:** Allows numeric comparisons between `INTEGER` and `FLOAT`; disallows illegal operations like `STRING > INTEGER`.

Context-Aware Error Diagnostics

```
[SEMANTIC ERROR] at Line 1, Column 8:
Column 'salary' does not exist in table 'students'.
  --> SELECT salary FROM students;
             ^
  HINT: Available columns in 'students':
        id, name, age, branch, cgpa

-----------------------------------------------
[SEMANTIC ERROR] at Line 1, Column 38:
Operator '>' cannot be applied to STRING and INTEGER.
  --> SELECT name FROM students WHERE name > 50;
                                         ^
```

9. Relational Algebra Intermediate Representation (IR)

Translating decorated AST into a decoupled logical query execution plan

Relational Operators (`planner/plan.py`)

- `ScanNode`: Base relation access operator.
- `FilterNode`: Selection operator (σ) filtering tuples by predicate.
- `ProjectNode`: Projection operator (π) discarding non-requested attributes.
- `SortNode`: Ordering operator (τ) sorting dataset tuples.
- `LimitNode`: Cardinality operator truncating output rows.
- **Decoupling Benefit:** Isolates query parsing from evaluation, enabling algebraic optimizations in Phase 3.

Generated Logical Plan Tree (IR)

```
\-- PROJECT [name, cgpa]
    \-- FILTER [cgpa >= 8.5 AND age > 20]
        \-- SCAN [students]

Formal Relational Algebra Expression:
π_{name, cgpa} ( σ_{cgpa ≥ 8.5 ∧ age > 20} ( SCAN(students) ) )
```

10. In-Memory Execution Engine & Results

Runtime interpreter evaluating the logical plan over in-memory datasets

Engine Execution Architecture

- **Tree-Walking Interpreter:** Recursively executes logical plan operators over in-memory relational dictionaries.
- **Predicate Evaluation:** Expression tree evaluator supporting comparison operators and boolean short-circuit logic (`AND`, `OR`).
- **Performance Telemetry:** Automatically measures rows scanned, rows filtered, and rows returned.
- **ASCII Table Formatter:** Renders dynamically sized tabular result sets.

Canonical Query Execution Result

```
Query:
SELECT name, cgpa FROM students
WHERE cgpa >= 8.5 AND age > 20;

+--------+--------+
| NAME   | CGPA   |
+--------+--------+
| Asha   | 8.7    |
| Rahul  | 9.1    |
| Pooja  | 8.9    |
| Ananya | 9.4    |
+--------+--------+
4 row(s) in set (Scanned: 7, Filtered: 4)
```

11. Comprehensive Testing Matrix & Verification

Automated verification matrix and PyUnit test harness results (100% pass rate)

Test ID	Category	Input Query / Condition	Expected Output	Status
T-01	Basic SELECT	`SELECT name FROM students;`	7 rows returned	PASS
T-03	WHERE Filter	`SELECT name FROM students WHERE age > 20;`	4 rows filtered	PASS
T-04	Logical AND	`SELECT name, cgpa FROM students WHERE cgpa >= 8.5 AND age > 20;`	4 rows filtered	PASS
T-07	ORDER BY	`SELECT name, cgpa FROM students ORDER BY cgpa;`	Sorted ascending	PASS
T-09	Lexical Error	`SELECT name @ students;`	LexicalError raised	PASS
T-10	Syntax Error	`SELECT FROM students;`	SyntaxError raised	PASS
T-13	Semantic Error	`SELECT salary FROM students;`	SemanticError raised	PASS
T-14	Type Mismatch	`SELECT name FROM students WHERE name > 50;`	SemanticError raised	PASS

✓ Total Tests: 16 Formal Matrix Tests + 20 PyUnit Test Cases | Passed: 100% | Execution Time: 0.004s

12. Live Demonstration Scenarios (Demos 1 to 5)

Pre-configured operational scenarios ready for interactive evaluation

Demos 1 to 4: Functional Scenarios

Demo 1: Basic Projection

`SELECT name FROM students;` → Validates projection without filter.

Demo 2: Relational Filtering

`SELECT name, age, cgpa FROM students WHERE age > 20;` → Evaluates condition operator.

Demo 3: Semantic Diagnostics

`SELECT salary FROM students;` → Catches missing attribute with column hints.

Demo 4: Syntactic Diagnostics

`SELECT FROM students;` → Catches missing projection column.

Demo 5: Deep-Dive Pipeline Inspection

Canonical Query: `SELECT name, cgpa FROM students WHERE cgpa >= 8.5 AND age > 20;`

- **Stage 1 (Lexer):** Full token stream with line and col markers.
- **Stage 2 (Parser):** Hierarchical AST visual tree output.
- **Stage 3 (Semantic):** Symbol binding & type checks confirmed.
- **Stage 4 (Planner):** Relational algebra IR execution tree.
- **Stage 5 (Engine):** Formatted result set with execution metrics.

13. Conclusion & Phase 3 Roadmap

Review 2 accomplishments and preparation for final submission

Review 2 Accomplishments

- Transitioned from an isolated lexer prototype to a complete, operational compiler pipeline.
- Demonstrated concrete practical implementations of Lexer, Parser, AST, Symbol Table, Semantic Analyzer, IR Planner, and Execution Engine.
- Constructed an automated verification test suite and multi-case demonstration runner.

Phase 3 Planned Deliverables

- **Query Optimizer:** Rule-based algebraic rewrites (predicate pushdown, constant folding, tautology elimination).
- **Language Extensions:** Multi-table relational JOINs and aggregate functions (`COUNT`, `AVG`, `SUM`).
- **Interactive UI:** GUI / Web visualizer for query pipeline inspection.
- **Performance Benchmarks:** Comparative execution timing across varying dataset scales.